FOR COMPLETE BEGINNERS

One count. Every object.

Complete Modern Java, episode 24: static and constants, in Java and TypeScript

> Values and methods that belong to the class itself, a constant that cannot change, and the mistake static invites.

Every product has its own name and price. How many products have been made, and the most a price can be, belong to no single product.

static means one for the whole class, shared by every object. static final makes a constant.

In this episode

- static fields
- static methods
- static final constants
- Naming constants
- Non-static from a static context
- TypeScript static readonly

Coming next

- Interfaces (episode 25)

At the time of recording: Java 25, Node 24 LTS, IntelliJ IDEA 2026.1.3.

01 What each run printed

getCreatedCount(), two products, again	`0, 2`
MAX_PRICE_CENTS	`100000`
isValidPrice(150); isValidPrice(150_000)	`true, false`
Product.MAX_PRICE_CENTS = 5;	`compile error`
Product.getName()	`compile error`
counter.ts	`0, 2, 100000`

Every row was run, not predicted.

02 One counter for the class

A static field exists once for the whole class, not once per object. The constructor adds one after the price is checked, so it counts products successfully made while the program runs.

Shared state is shared by every piece of code: change it in one place and it changes for everyone.

Product.java

```
class Product {
    static final int MAX_PRICE_CENTS = 100_000;
    private static int createdCount = 0;

    Product(String name, int priceCents) {
        this.name = name;
        setPriceCents(priceCents);
        createdCount++;
    }

    static int getCreatedCount() {
        return createdCount;
    }

    static boolean isValidPrice(int priceCents) {
        return priceCents >= 0 && priceCents <= MAX_PRICE_CENTS;
    }
    // fields, getters and setPriceCents as in episode 17
}
```

> Counter.java printed 0, then 2, then 100000, true, false and Bread.
>
> Run on a Mac, 2026-11-07: Java 25.0.4, Node 24.22.0, TypeScript 7.0.2.

03 Two things the compiler refuses

final means a variable can never be given another value. Constants are written in capitals with underscores.

A static method has no object of its own. Product.getName() asks which product? A name belongs to an object: bread.getName().

Two compile errors

```
Product.MAX_PRICE_CENTS = 5;
// cannot assign a value to static final variable MAX_PRICE_CENTS
IO.println(Product.getName());
// non-static method getName() cannot be referenced from a static context
```

> Both messages are copied from javac.
>
> Run on a Mac, 2026-11-07: Java 25.0.4, Node 24.22.0, TypeScript 7.0.2.

04 TypeScript static readonly

readonly is TypeScript's version of final, enforced by the checker. node still runs the assignment.

counter.ts

```
class Product {
    static readonly MAX_PRICE_CENTS = 100_000;
    static createdCount = 0;
    // constructor: Product.createdCount++;
}
Product.MAX_PRICE_CENTS = 5;   // TS2540 from tsc; node prints 5
```

> tsc quiet without the assignment; 0, 2, 100000. With it: TS2540, and node printed 5.
>
> Run on a Mac, 2026-11-07: Java 25.0.4, Node 24.22.0, TypeScript 7.0.2.

static, in one look

1. One value for the whole class? static.
2. A value that must never change? static final, in capitals.
3. A method that needs no object? static, called on the class.
4. Needs one object's fields? Call it on that object.
5. Challenge: make a 150_000 product in a try. The count after? 0.

Everything else

Site
https://code-with-sam-dev.github.io

YouTube
https://www.youtube.com/@CodewithSam-Dev

LinkedIn
https://www.linkedin.com/company/code-with-sam-dev

X
https://x.com/CodeWithSamDev

TikTok
https://www.tiktok.com/@codewithsamdev

Instagram
https://www.instagram.com/codewithsamdev

Facebook
https://www.facebook.com/codewithsamdev

GitHub
https://github.com/code-with-sam-dev

Buy me a coffee
https://buymeacoffee.com/codewithsam

Buy me sadza
https://buymesadza.com/codewithsam

The code and the challenge files: complete-modern-java
https://github.com/code-with-sam-dev/complete-modern-java

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Java is a trademark of Oracle. Node.js is a trademark of the OpenJS Foundation. TypeScript is a trademark of Microsoft. IntelliJ IDEA is a trademark of JetBrains. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-11-07 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.